

CBO OS

Engineering handoff: product definition, architecture, data model, agent loops, interfaces, requirements and delivery plan.

LOCAL-FIRST	AGENTIC BD	HUMAN CONTROL	INTERFACE-READY
-------------	------------	---------------	-----------------

PRODUCT PRINCIPLE

Turn a commercial objective into qualified human conversations, and turn those conversations into measurable outcomes.

Initial deployment: local-first, single operator, no authentication. Architecture must remain interface-ready for external systems such as AWEEDINARY and Good Paper Agent OS.

1. Executive brief

CBO OS is a local-first agentic business-development operating system. Its unit of work is an **Endeavour**: a bounded commercial objective such as generating £3,000 in consulting revenue, acquiring 10 liquidity providers, finding five pilot customers, securing grant conversations, or generating qualified interviews.

The product is deliberately separate from project/operations management. It does not own engineering milestones, delivery planning, or product execution. It owns commercial discovery and execution: target audiences, offers, positioning, prospect research, qualification, outreach, follow-ups, pipeline, feedback and strategy adaptation.

The system should behave like a persistent digital Chief Business Development Officer rather than an email sequencer. Every day it determines what commercial work is due, researches new targets, prepares or executes approved actions, processes responses, updates the pipeline and reports what it learned.

Scope boundary

CBO OS owns	Commercial objectives, ICPs, offer hypotheses, positioning, prospecting, qualification, outreach, follow-up, conversation state, pipeline, commercial signals, experiments, performance and recommendations.
CBO OS does not own	Product specs, engineering tasks, code delivery, operational project tracking, treasury/accounting, or general company administration.
External systems	AWEEDINARY (CTO/org builder) and Good Paper Agent OS (operations) remain independent. CBO OS must expose clean import/export and event interfaces so data can be connected later without making integration a v1 workstream.

2. Product model

The core object is the Endeavour. An Endeavour is not a project. It is a commercial campaign with a measurable outcome, a deadline or horizon, a target market and a defined outreach capacity.

Example

```
ENDEAVOUR: £3K Solidity Consulting Sprint objective.value = 3000 GBP horizon = 60 days minimum_deal = 300 GBP channels = [email, linkedin/manual] daily_new_target = 10 daily_followup_target = 8 approval_mode = draft_before_send status = active
```

Endeavour lifecycle

Objective → Market → Position → Discover → Qualify → Engage → Follow-up → Learn → Close. The loop is continuous: replies, objections and conversion data update targeting and messaging hypotheses.

Required Endeavour modules

- **Objective:** outcome, value, unit, deadline, constraints, minimum acceptable deal and success criteria.
- **ICP / Segments:** target company/person types, exclusions, signals, pain hypotheses and priority.
- **Offers:** service/product proposition, price model, evidence, CTA and objection handling.
- **Positioning:** profile copy, proof assets, portfolio links, credibility gaps and recommended changes.
- **Prospects:** company, person, role, source, trigger, evidence, qualification score and reason to contact now.
- **Outreach:** channel, message version, personalization evidence, approval state, send state and timestamps.
- **Pipeline:** discovered, researched, qualified, contacted, replied, meeting, proposal, won, lost, nurture.

- **Intelligence:** response patterns, objections, segment performance, offer performance, experiments and recommendations.

3. User experience

V1 is single-user and local. No login, teams, billing or permissions system is required. On launch, the user lands directly on the command dashboard.

Primary screens

- **Command Dashboard:** all active Endeavours, objective progress, pipeline value, actions due, critical replies and daily briefing.
- **Endeavour Overview:** objective, current strategy, funnel, recent learning, key numbers and today's queue.
- **Prospects:** searchable/filterable research table with evidence, score, stage, next action and contact history.
- **Pipeline:** clean stage view; prioritize information density over decorative CRM conventions.
- **Inbox / Conversations:** relevant commercial threads, classifications, proposed replies, next action and outcome.
- **Strategy:** ICPs, offers, positioning, messaging hypotheses, active experiments and version history.
- **Research:** discoveries awaiting qualification, research sources, trigger events and rejected targets.
- **Daily Brief:** what happened, what matters, what the agent proposes today, what needs human approval.
- **Settings / Interfaces:** model/provider, data paths, email connector configuration, autonomy settings, import/export and interface keys.

4. Daily operating loop

The scheduler is the heartbeat. Each active Endeavour has daily activity targets and next-action rules. A run must be idempotent and auditable.

```
DAILY RUN 1. Load active Endeavours + constraints. 2. Read yesterday's actions, replies and outstanding follow-ups. 3. Process inbound commercial signals. 4. Recalculate pipeline and performance metrics. 5. Research enough candidates to satisfy today's qualified-prospect target. 6. Score and reject/accept candidates with evidence. 7. Generate today's new outreach and follow-up queue. 8. Draft personalized messages using only supported evidence. 9. Execute only actions allowed by autonomy policy. 10. Escalate replies, ambiguity, high-value opportunities and exceptions. 11. Update learning: objections, segment/offer/message performance. 12. Produce Daily Brief and persist complete run log.
```

The agent must never fabricate personalization. If a claim about a prospect cannot be supported by stored research evidence, the message should not contain it.

5. Autonomy and approvals

Autonomy must be explicit per Endeavour and per action class. Start conservative. Research, scoring, CRM updates, analytics and drafting can be autonomous. Sending should default to approval-required until the user deliberately changes policy.

Level 0 - Observe	Research and analyze only. No outbound actions.
Level 1 - Draft	Research, qualify, create queues and draft messages/replies. Human sends.
Level 2 - Guarded send	May send pre-approved message classes within daily caps; novel copy and sensitive replies require approval.

Hard controls

- Daily and weekly send caps by channel.
- Do-not-contact list and domain/person suppression.
- No duplicate outreach to the same person unless a valid follow-up is due.
- Quiet hours and channel-specific cadence.
- Human approval for price concessions, contractual commitments, sensitive claims or unusual replies.
- Complete audit log of agent reasoning summaries, evidence references, drafts, approvals and actions.

6. Suggested architecture

Prefer a boring, inspectable local architecture over unnecessary distributed infrastructure. A TypeScript full-stack application is appropriate. The exact framework may change, but keep domain logic independent of UI and model provider.

```
[Web UI] | [Application API] |-- Endeavour Service |-- Prospect / CRM Service |-- Outreach Service
|-- Conversation Service |-- Research Service |-- Intelligence Service |-- Scheduler / Job Runner
|-- Approval Service |-- Interface Gateway | [Agent Orchestrator] ---- [Claude / LLM Adapter] | [Tool
Adapters] |-- Web research |-- Email |-- Import/export |-- Future channel connectors | [Postgres or
SQLite for local v1] |-- structured state |-- event log |-- evidence |-- run history | [Local
file/object store] |-- generated briefs |-- exports |-- attachments
```

Recommended implementation choices

Frontend	Next.js/React + TypeScript. Dense desktop-first command UI; responsive but not mobile-first.
Backend	Next.js server/API or a small Node service. Keep services modular so workers can separate later.
Database	PostgreSQL if already available; SQLite is acceptable for a zero-friction local prototype. Use migrations from day one.
Jobs	Simple persisted job table + worker/cron loop initially. Do not introduce Kafka/queues unless scale requires it.
LLM	Provider adapter with Claude as primary. Prompts are versioned assets; outputs use strict schemas and validation.
Research	Tool adapter returning source URL/reference, timestamp, excerpt/claim and confidence. Evidence is first-class data.
Email	Connector abstraction. Read thread metadata, draft/send according to autonomy, map messages to prospects and Endeavours.
Observability	Structured logs, run IDs, tool-call summaries, failure state, token/cost accounting and action audit.

7. Data model

Use normalized relational state for operational truth and JSON only where flexibility is genuinely useful. Every important object should carry `created_at`, `updated_at` and `source/provenance` where relevant.

```
endeavours(id, name, status, objective_json, horizon, autonomy_level, daily_new_target,
daily_followup_target, constraints_json) segments(id, endeavour_id, name, definition, priority,
hypothesis, status) offers(id, endeavour_id, name, proposition, pricing_json, cta, status)
prospects(id, endeavour_id, company_id, person_id, segment_id, stage, qualification_score,
score_reason, next_action_at, owner) companies(id, name, domain, description, metadata_json)
people(id, company_id, name, role, email, linkedin_url, metadata_json) evidence(id, entity_type,
entity_id, source_type, source_ref, claim, captured_at, confidence) triggers(id, prospect_id, type,
description, evidence_id, detected_at) messages(id, prospect_id, endeavour_id, channel, direction,
thread_ref, body, template_version, approval_state, sent_at) activities(id, prospect_id,
endeavour_id, type, status, due_at, completed_at) opportunities(id, prospect_id, endeavour_id,
value, currency, stage, probability_user_defined, expected_close, outcome_reason) experiments(id,
endeavour_id, dimension, variant_a, variant_b, status) insights(id, endeavour_id, type, statement,
evidence_json, confidence) daily_runs(id, endeavour_id, run_date, status, metrics_json, summary)
approvals(id, endeavour_id, action_type, payload_json, status, decided_at) events(id, source_system,
event_type, entity_type, entity_id, payload_json, occurred_at, schema_version)
```

8. Qualification and intelligence

Qualification should be transparent. Do not hide target selection behind an opaque LLM judgment. The LLM may interpret evidence, but the application stores the factors and explanation.

A useful initial score can combine ICP fit, commercial trigger strength, likely pain, decision-maker relevance, evidence quality, timing and contactability. Scores are prioritization aids, not truth. The user must be able to inspect why a prospect ranked highly.

Learning engine

- Track positive reply rate, meeting rate, proposal rate, win rate and revenue by segment, offer, source, trigger and message version.
- Extract objections and classify them without overwriting the original reply.
- Detect repeated requests or objections as commercial signals.
- Recommend changes only when evidence is visible: e.g. 'launch-stage protocols produced 5/22 positive replies vs 1/41 for general protocols.'
- Maintain experiment history so the system does not continually reinvent strategy from tiny samples.

9. Interface contract: essential, not a v1 integration project

CBO OS must be independently useful, but its boundary should be designed so AWEEDINARY, Good Paper Agent OS and future systems can consume and emit structured data. Build the interface seam now; do not spend the project integrating those systems.

Minimum interface

```
REST/JSON or local HTTP API GET /api/v1/endeavours GET /api/v1/endeavours/:id/summary GET
/api/v1/endeavours/:id/prospects GET /api/v1/endeavours/:id/insights GET
/api/v1/endeavours/:id/events?since=... POST /api/v1/events POST /api/v1/signals POST /api/v1/import
Exports: JSON + CSV Event schema: versioned Webhook support: optional later Auth: none for local v1;
design middleware boundary so auth can be added later.
```

Canonical cross-system events

- **product.milestone.completed** - external operational system reports something commercially relevant is now live.
- **product.capability.changed** - new/changed capability can affect messaging or targeting.

- **commercial.signal.detected** - repeated request, objection, market pattern or customer need discovered by CBO OS.
- **commercial.opportunity.created/updated** - useful for executive or operational dashboards.
- **commercial.commitment.made** - a promise made to a prospect that another system may need to know.

10. Email and conversation handling

Email is a tool adapter, not the database of record. Persist stable external IDs and enough metadata to reconstruct state without unnecessarily duplicating entire mailboxes.

Inbound processing should map thread → person → prospect → Endeavour; classify reply intent; extract commitments, objections and requested next steps; update pipeline; create due actions; and prepare a response where appropriate. Uncertain mapping should go to a review queue.

Do not auto-send a response merely because it can be drafted. Sending is governed by the Endeavour autonomy policy.

11. Daily Brief specification

The Daily Brief is the executive surface and should be generated from stored state, not free-form model memory.

- Objective progress and remaining gap.
- Pipeline value and stage movement since previous run.
- New qualified prospects found; new outreach due; follow-ups due.
- Replies received, meetings/proposals/wins/losses.
- Critical opportunities requiring immediate attention.
- What changed in the market or prospect set.
- What the system learned, with supporting numbers/evidence.
- Today's proposed actions and any approvals required.
- Failures or coverage gaps: e.g. research source unavailable, email sync failed, target quota not met.

12. Brand and visual system

The brand should feel like **institutional finance software crossed with a restrained cyberpunk command terminal**. It is not a neon hacker dashboard, not generic SaaS purple, and not a sales CRM clone.

Theme	Light-first. Warm/cool off-white canvas, charcoal text, pale mineral-grey panels, subtle green/cyan machine accents.
Palette	Canvas #F4F7F5; paper #FFFFFF; ink #111817; muted #5E6A66; grid #D9E2DE; signal green #16A36A; deep green #087A4B; optional cyan #5CC8B2. Red/amber reserved for actual state.
Typography	Clean grotesk/sans for UI (Inter/Geist-like). Monospace only for IDs, timestamps, system states, metrics and machine annotations.
Geometry	Sharp or lightly rounded 4-6px corners. Fine 1px rules. Avoid bubbly cards and excessive pills.

Texture	Very subtle matrix/grid/data texture only in atmospheric areas. No literal Matrix rain dominating the interface.
Motion	Quiet, functional: scanning line, live status pulse, number transitions, subtle queue movement. No gratuitous parallax.
Voice	Precise, commercial, unemotional. '12 qualified / 10 required' rather than motivational copy.

13. Non-functional requirements

- Local-first and usable without account creation.
- All agent actions reproducible from persisted inputs and versioned prompts where practical.
- Schema validation on model outputs; invalid outputs fail safely.
- No silent send failures; outbound action state must be explicit.
- No unsupported prospect claims or fabricated research.
- Fast dashboard load from database; agent work occurs asynchronously.
- Secrets live in environment/keychain, never database plaintext where avoidable.
- Backups/export must be trivial: database + local files + JSON export.
- System should survive restart mid-run and resume or clearly mark interrupted jobs.

14. V1 acceptance criteria

- Create, edit, pause and archive an Endeavour with objective, horizon, ICP, offers and daily targets.
- Store and manage prospects with evidence, qualification score, stage and next action.
- Run a daily cycle that creates a deterministic work queue and Daily Brief.
- Research prospects through a replaceable adapter and persist evidence/provenance.
- Generate personalized outreach drafts tied to evidence.
- Track outbound/inbound interactions and follow-up due dates.
- Support approval queue and at least Levels 0-1 autonomy cleanly.
- Show funnel and core performance by segment/offer/message version.
- Extract and surface objections/insights from replies.
- Expose versioned JSON API + JSON/CSV export + generic inbound event endpoint.
- No login required for local deployment.
- UI conforms to the supplied brand system and does not look like a generic CRM template.

15. Suggested build sequence

Phase 1 - Skeleton: app shell, DB schema/migrations, Endeavour CRUD, prospects, pipeline, event log, settings.

Phase 2 - Agent core: Claude adapter, structured prompts, research/evidence, qualification, daily runner, queue, Daily Brief.

Phase 3 - Outreach: message drafting, approvals, conversation mapping, email adapter, follow-up engine.

Phase 4 - Intelligence: experiments, objection extraction, performance cuts, recommendations, commercial signals.

Phase 5 - Interface seam: v1 API, events/signals, import/export. Validate with mocked AWEEDINARY and Agent OS payloads; do not build full integrations.

16. Engineering instruction to Claude

Treat this document as the product contract. Before implementation, inspect the existing repository and produce a concise gap analysis and proposed file/module plan. Do not invent integrations, authentication, billing or enterprise abstractions that are not required. Prefer a small working system with clean domain boundaries and persistent state over a broad demo full of pseudo-data.

Any placeholder data used during development must be clearly marked as fixtures and removable in one operation. The finished local app should start empty and guide the user to create the first Endeavour.